Github Link - https://github.com/TammyBriggs/alu-logo-verification-dapp

Demo Video - Videos | Library | Loom - 17 July 2026

Abstract

This project presents a decentralized application (dApp) that bridges the gap between secure blockchain smart contracts and end-user accessibility. By providing an intuitive web interface for logo authentication and tokenized asset management, the dApp solves the problem of brand counterfeiting and provides a transparent, automated solution for distributing ownership shares within the ALU community.

Introduction

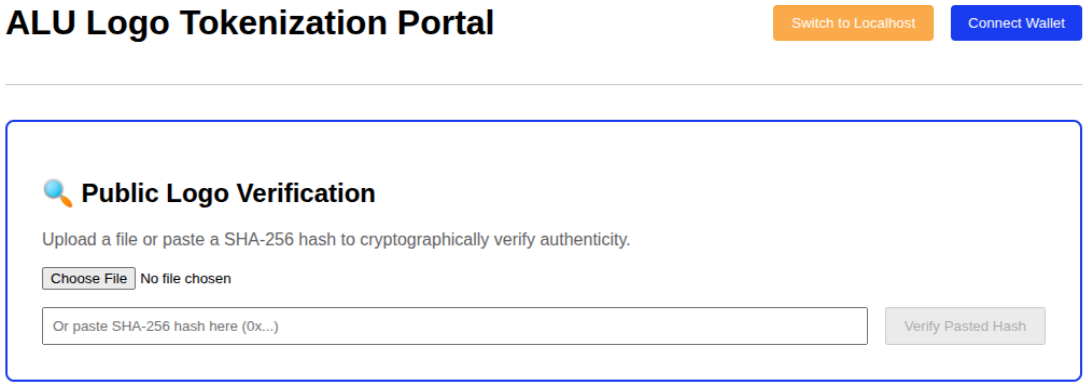
While smart contracts provide a permanent, immutable ledger for registering the ALU logo, they remain inaccessible to non-technical users who are uncomfortable using terminal-based tools like Hardhat. This dApp fills that "usability gap" by serving as a graphical front door. It transforms raw blockchain data into a user-friendly experience, allowing students, faculty, and administrators to perform complex cryptographic verifications and financial transactions through a standard web browser.

Architecture Overview

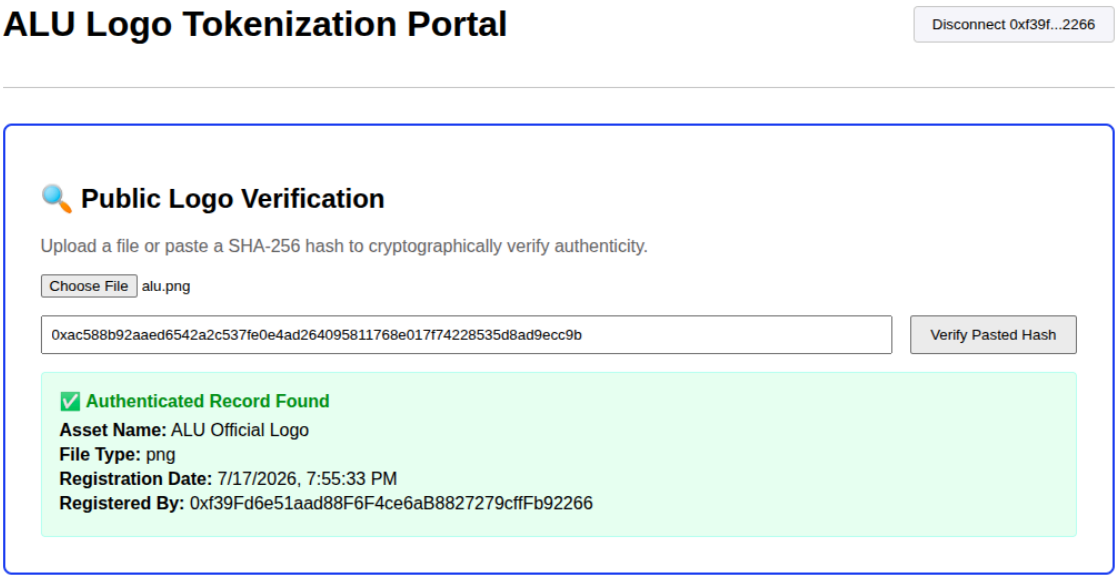
The application follows a three-tier Web3 architecture:

1. **Frontend (React/Vite): Manages the user interface and browser-side logic, such as file hashing.**
2. **Web3 Connector (ethers.js): Acts as the communication layer. It uses the `BrowserProvider` for wallet interactions (signed transactions) and a `JsonRpcProvider` for public data reads (gas-free verification).**
3. **Smart Contracts (Solidity): The backend residing on the blockchain, containing the core logic for asset registration, integrity checks, and ERC-20 token distribution. *Communication flow:* User interactions trigger functions in the frontend, which are encoded using the contract ABIs and sent via the user's wallet or a public RPC node to the Ethereum Virtual Machine (EVM).**

Feature Walkthrough



- **Wallet Connection & Network Management:** The application header acts as the secure entry point for the dApp. It intelligently detects the user's Web3 provider (like MetaMask) and verifies their network connection. If a user is on the wrong network, it dynamically prompts them to switch to the local Hardhat node. Once connected, it displays their truncated wallet address and provides a clean "Disconnect" button that securely refreshes the application state.



- **Public Logo Verification:** This gas-free feature is accessible to all users, even those without a connected wallet. Users can verify the authenticity of a file by uploading an image or pasting a SHA-256 hash directly into the input field. The frontend instantly queries the blockchain registry and, if a match is found, displays the official on-chain metadata, including the asset's name, file type, and exact registration timestamp.

📈 **ALUT Token Dashboard**

Total Supply: 1000000 ALUT

Your Balance: 990000 ALUT

Your Fractional Share: 99%

Network Example Wallets

`0x7099...79C8` **1%** share

`0x3C44...93BC` **0%** share

`0x90F7...b906` **0%** share

- **Token Dashboard:** The dashboard provides transparent, real-time metrics for the ALUT token economy. It fetches data directly from the ERC-20 smart contract to display the total circulating supply alongside the connected user's exact balance and fractional ownership percentage. It also tracks the holdings of other key network wallets to ensure complete economic visibility for stakeholders.

📝 Register New Asset / Logo Variant

Asset Name:

Yummy Food

Upload File to Hash:

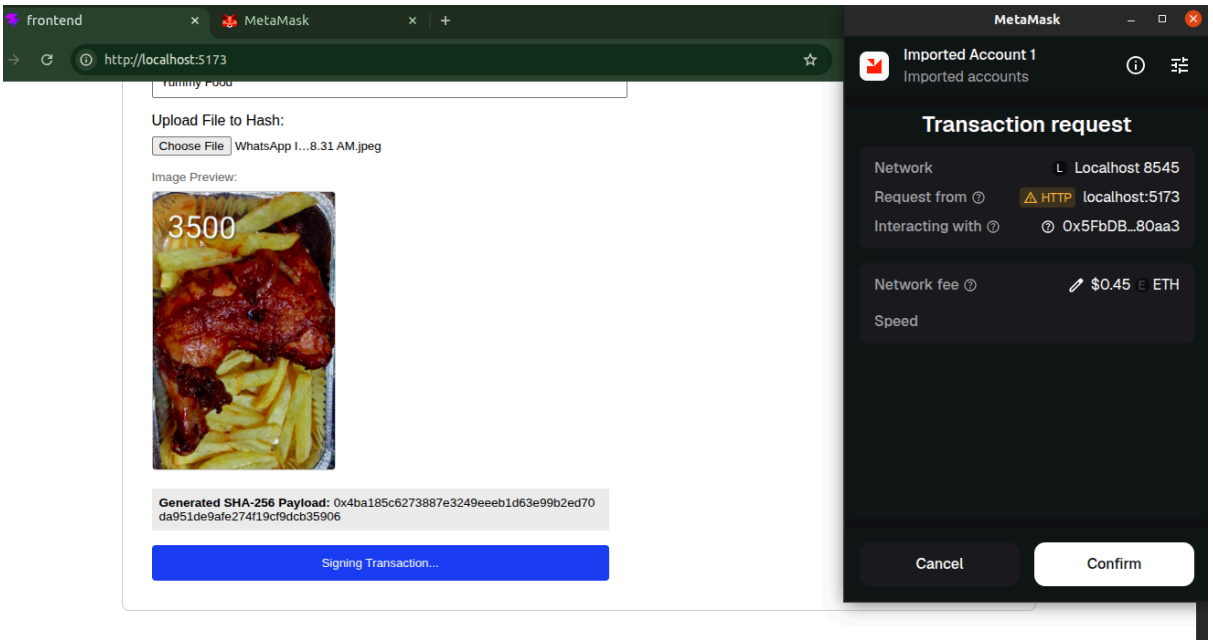
Choose File | WhatsApp I...8.31 AM.jpeg

Image Preview:

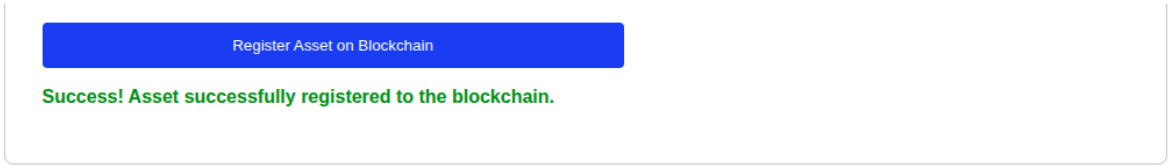
Generated SHA-256 Payload: 0x4ba185c6273887e3249eeeb1d63e99b2ed70da951de9afe274f19cf9dcb35906

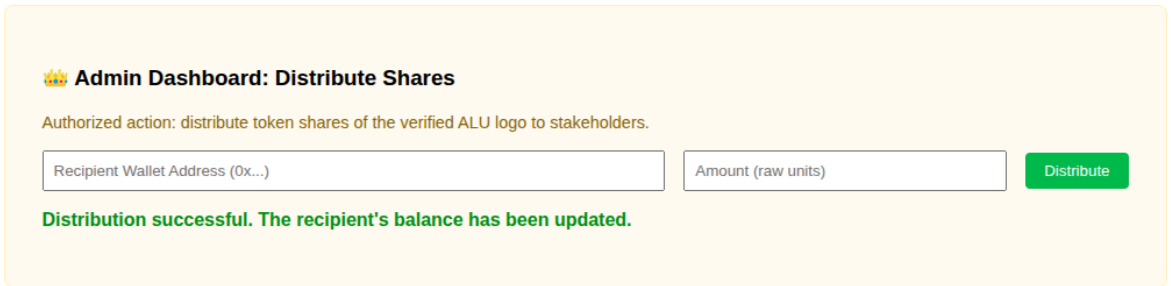
Register Asset on Blockchain

- **Asset Registration Form:** Authorized users can tokenize new official brand assets through this interface. To prioritize privacy and decentralization, when a user selects a file, the frontend generates a local image preview and computes the cryptographic SHA-256 hash entirely within the browser. The raw file never leaves the user's machine; only the mathematical payload is sent to the blockchain.



- **Web3 Transaction Signature (MetaMask):** Whenever a user interacts with a state-changing function, such as registering a new logo hash or transferring tokens, the dApp delegates the authorization to the user's secure Web3 wallet. This ensures that the application cannot execute unauthorized state changes or consume gas fees without the explicit cryptographic signature of the keyholder.





- **Admin Dashboard & Token Distribution:** This administrative section implements strict access control by conditionally rendering only if the connected wallet is the official contract owner. It provides a straightforward interface to distribute ALUT ownership shares to other university stakeholders. The form includes robust frontend

validation to prevent transactions involving invalid wallet addresses or zero-value amounts.

Wallet and Web3 Integration

What is an ABI, and why does the frontend need it to talk to a smart contract? An **Application Binary Interface (ABI)** is a JSON-formatted template that lists a smart contract's functions, parameters, data types, and return behaviors. Because smart contracts are compiled into bytecode on the blockchain, the frontend cannot natively decipher how to format requests. The frontend needs the ABI as a translation guide to correctly encode JavaScript/TypeScript inputs into bytecode transactions and decode blockchain responses back into human-readable text.

What is the difference between a read-only call and a transaction?

- **Read-only Call:** Invokes a smart contract function configured with the `view` or `pure` modifiers. It reads existing data from the blockchain without modifying the state. It does not require a cryptographic wallet signature, does not change the blockchain ledger, and executes instantly with zero gas cost.
- **Transaction:** Invokes a state-changing function that updates, deletes, or writes new information onto the blockchain ledger. It requires a cryptographic signature from a user's Web3 wallet to authorize the state change, consumes computational gas fees paid in native cryptocurrency (e.g., ETH), and must be mined into a block to be finalized.

Why does the user need to sign transactions with their wallet rather than the application signing on their behalf? In decentralized architectures, the application frontend has no custody or access to the user's private keys. Forcing the transaction signature to happen strictly inside the isolated environment of a secure Web3 wallet extension (like MetaMask) ensures that the application cannot perform unauthorized state changes, drain assets, or forge actions without the explicit, cryptographic approval of the keyholder.

How does wallet connection work, and what is ethers.js doing?
Wallet connection relies on the dApp requesting access to the window.ethereum object (injected by the wallet extension). When the user clicks "Connect," ethers.js uses the BrowserProvider to initiate the eth_requestAccounts JSON-RPC call. Once the user approves this in their wallet, ethers.js provides the application with the user's public address and acts as an interface layer that translates JavaScript function calls into hex-encoded transaction data, which the wallet then signs and transmits to the blockchain.

Verification Page / Hashing

Why is it important to generate the hash in the browser rather than sending the file to a server? Generating the hash in the browser guarantees zero-knowledge privacy and decentralization. The image file never leaves the user's machine, eliminating the risk of data

interception, central server downtime, or storage costs. It ensures the dApp remains a true Web3 product, relying solely on client-side logic and blockchain infrastructure.

What happens at the smart contract level when a user tries to register a hash that already exists? When a duplicate hash is submitted, the smart contract's state requirement (typically evaluated via a `require` statement checking a `mapping`) fails. The EVM immediately reverts the transaction. All subsequent logic is cancelled, no state changes are written to the ledger, and the user forfeits a small amount of gas fees used to process the transaction up to the point of failure.

What does it mean for the user to approve a transaction in their wallet? What are they actually agreeing to? By signing the transaction, the user uses their private cryptographic key to explicitly authorize a specific state change on the blockchain. They are mathematically verifying their identity as the sender, agreeing to the exact parameters being passed to the contract (e.g., the hash and asset name), and agreeing to pay the necessary native token gas fees required by network validators to execute the transaction.

Explain how in-browser hashing works and why no server is needed.
In-browser hashing utilizes the SubtleCrypto.digest() method provided by the browser's native Web Crypto API. When a file is uploaded, the dApp reads the file data into an ArrayBuffer and passes it directly to this cryptographic function. Because the SHA-256 algorithm is performed locally by the user's CPU, the file never travels over the network. This removes the need for a backend server, as the "fingerprint" (hash) is generated entirely on the client side and then compared against the hash recorded on the blockchain via a simple view call.

Verification Page

Why does this page not require a wallet connection? What type of blockchain call is `verifyLogoIntegrity()`, and why does it cost no gas? This page functions without a wallet because it utilizes a local public RPC provider (`JsonRpcProvider`) to read the blockchain state instead of modifying it. `verifyLogoIntegrity()` is designated as a `view` function in Solidity. View functions only query the existing ledger data without initiating state changes; therefore, they require no computational gas fees and no cryptographic wallet signatures.

Give a real-world scenario in which someone would use this page. Who would use it and what problem does it solve? An external print design agency contracted by ALU to create official graduation banners might use this page. If they download multiple variations of the ALU logo from Google Images, they can upload them to this dApp. The app solves the problem of brand distortion by instantly identifying which specific file is mathematically proven to be the official, university-sanctioned version, preventing them from printing thousands of banners with an outdated or slightly modified counterfeit logo.

Why would someone checking a logo they downloaded from the internet trust the result shown by your dApp? The trust is derived from cryptography rather than a central

authority. SHA-256 hashing is deterministic and universally standardized; altering even a single pixel in an image produces a completely different hash. Because the authentic hash is stored immutably on the decentralized Ethereum blockchain, the user knows that the dApp is comparing their file's local mathematical footprint against an unalterable, permanent record registered by the official contract owner.

Token Dashboard

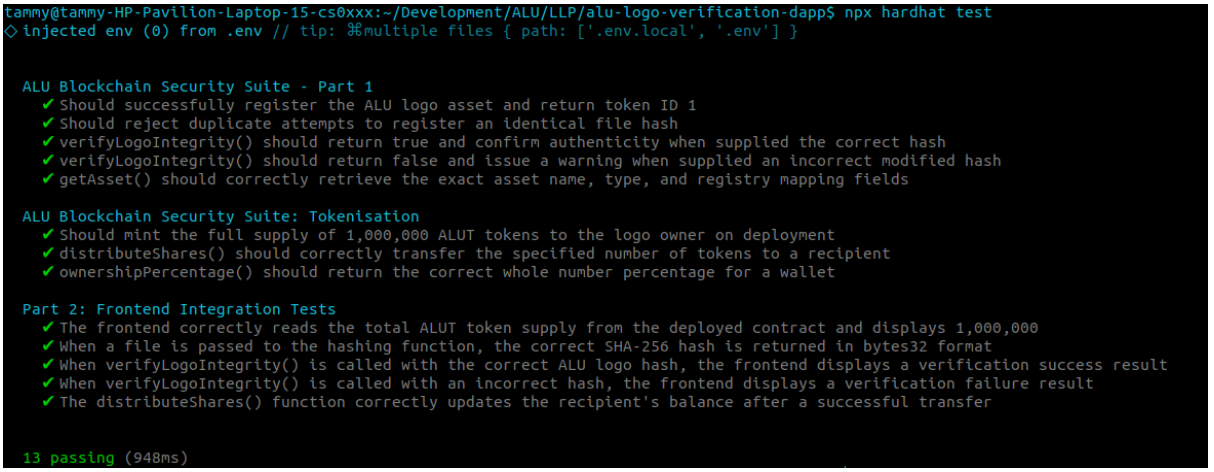
How does the onlyOwner modifier in the smart contract protect distributeShares()? What happens at the contract level if a non-owner wallet calls it? The `onlyOwner` modifier checks if the cryptographic signature of the `msg.sender` (the wallet executing the function) matches the specific owner address initialized during contract deployment. If a non-owner attempts the transaction, the EVM evaluation fails immediately. The transaction is fully reverted, throwing an `OwnableUnauthorizedAccount` error, and no state changes or token transfers are allowed to occur, securing the token supply from theft.

Give a specific example of who at ALU might receive ALUT tokens and what holding those tokens would mean for them in practice. The ALU Marketing and Communications department might receive a 40% distribution of ALUT tokens. Holding these tokens mathematically verifies their fractional ownership of the brand asset. In practice, they could use these tokens as a gateway to access restricted media kits or use them as collateral to prove their authorization when signing partnership agreements with external printing agencies.

What would need to change in the contract if ALU wanted shareholders to be able to vote on decisions proportional to their token holdings? The `ALULogoToken` contract would need to inherit advanced governance standards, such as OpenZeppelin's `ERC20Votes`. This extension adds logic to snapshot token balances at specific block numbers (checkpoints). Standard ERC-20 balances alone are vulnerable to "flash loan attacks," where a malicious actor borrows massive amounts of tokens, votes, and returns them in a single block. Checkpointing ensures voting power is determined historically, creating a secure, proportional DAO mechanism.

Explain what the dashboard shows and how distributeShares() is protected.
The dashboard provides a real-time ledger view of the token economy, specifically displaying the total supply of ALUT, the current balance of the connected user, and a breakdown of ownership percentages for the user and other example wallets. The distributeShares() function is protected by the onlyOwner modifier, which enforces an access control check at the smart contract level. When a transaction is submitted, the EVM checks if the msg.sender matches the owner variable stored in the contract; if the addresses do not match, the transaction is rejected before any tokens are moved, ensuring that only the authorized administrator can modify the ownership distribution.

Test Results



- **Summary:** All thirteen tests passed, including the eight original contract tests from Formative 1 and the five new frontend integration tests covering token supply, SHA-256 hashing accuracy, and integrity verification.

Conclusion

Through this project, I learned the critical importance of balancing decentralized security with user experience. I discovered how to bridge local browser-side cryptography with on-chain data to create a product that is both trustless and usable. With more time, I would improve the dashboard by adding a visual history of token distributions and integrating a subgraph to allow for faster, indexed queries of registered assets.